

ASSIGNMENT-9.2

AI ASSISTANT CODING

NAME: B. Akhira Nandhini

BATCH: 22

HTNO:2303A51516

Task Description -1 (Documentation – Function Summary Generation)

Task:

Use AI to generate concise functional summaries for each Python function in a given script.

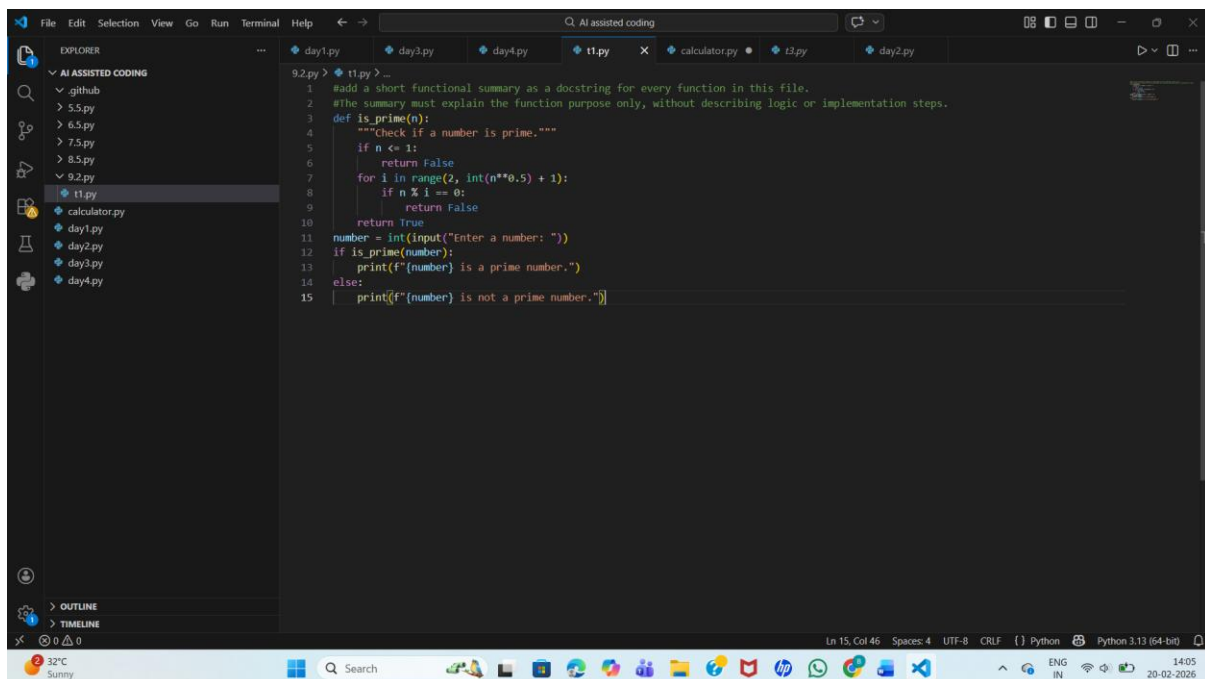
Instructions:

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details.

Expected Output -1:

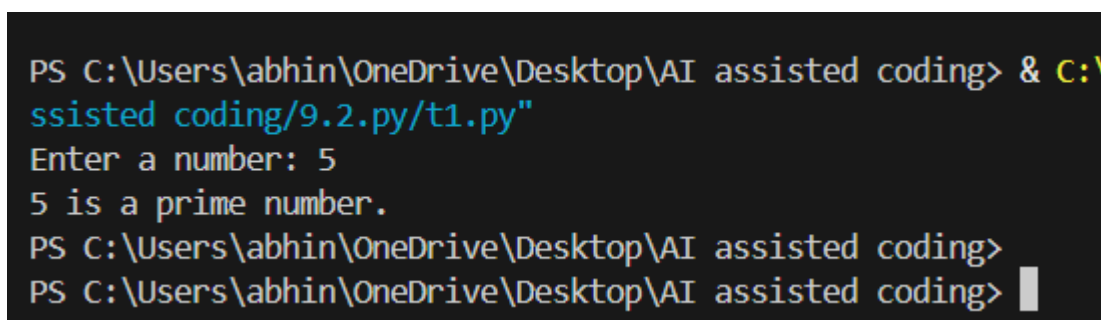
A Python script where each function contains a clear and concise summary explaining its purpose.

Code:



```
9.2.py > t1.py > ...
1 #add a short functional summary as a docstring for every function in this file.
2 #The summary must explain the function purpose only, without describing logic or implementation steps.
3 def is_prime(n):
4     """Check if a number is prime."""
5     if n <= 1:
6         return False
7     for i in range(2, int(n**0.5) + 1):
8         if n % i == 0:
9             return False
10    return True
11    number = int(input("Enter a number: "))
12    if is_prime(number):
13        print(f"{number} is a prime number.")
14    else:
15        print(f"{number} is not a prime number.")
```

Output:



```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\OneDrive\Desktop\AI assisted coding\9.2.py\t1.py
Enter a number: 5
5 is a prime number.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Observation:

AI-generated function summaries help improve code readability by clearly describing the purpose of each function. The summaries make it easier for developers to understand the program structure without analyzing the implementation details. This approach also supports faster maintenance and documentation consistency.

Task Description -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program.

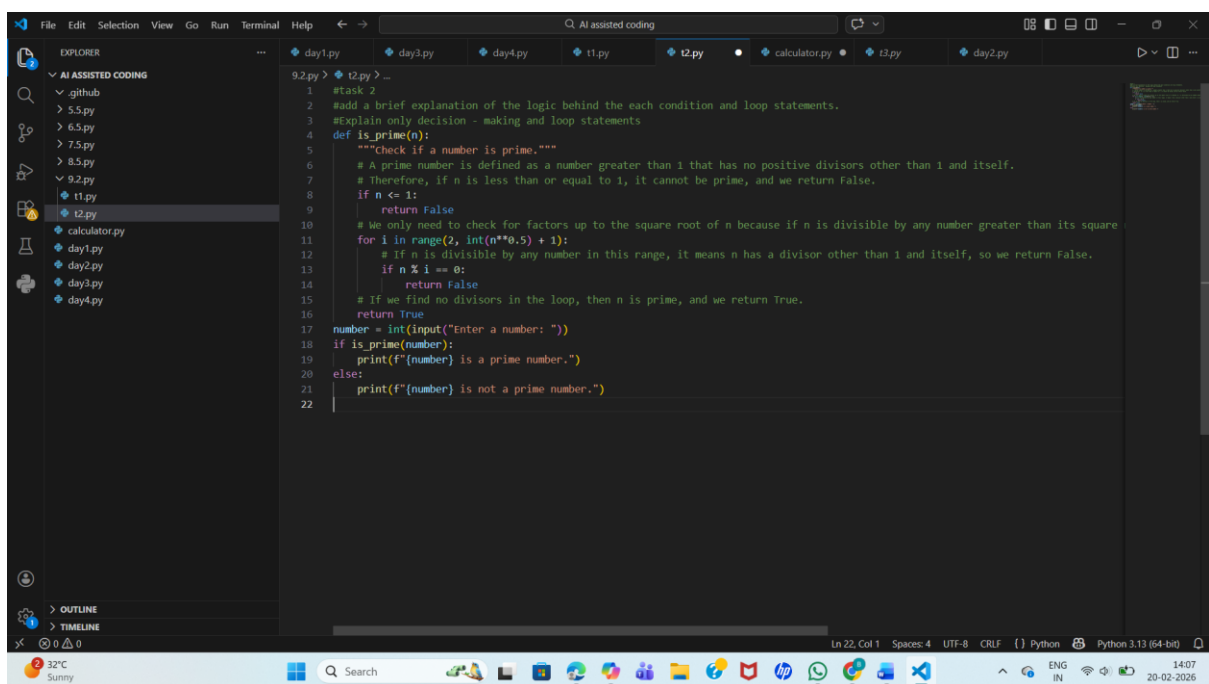
Instructions:

- Provide a Python program without comments.
- Instruct AI to explain only decision-making logic and loop behavior.
- Skip basic syntax explanations.

Expected Output -2:

Python code with clear explanations describing the logic of conditions and loops.

Code:

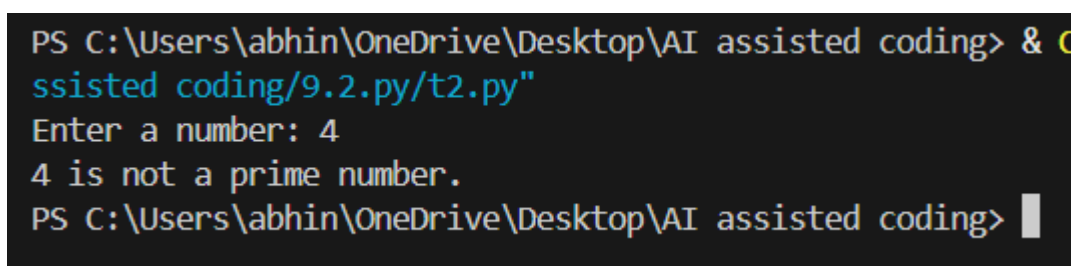


```

1 #task 2
2 add a brief explanation of the logic behind the each condition and loop statements.
3 #Explain only decision - making and loop statements
4 def is_prime(n):
5     """Check if a number is prime."""
6     # A prime number is defined as a number greater than 1 that has no positive divisors other than 1 and itself.
7     # Therefore, if n is less than or equal to 1, it cannot be prime, and we return False.
8     if n <= 1:
9         return False
10    # We only need to check for factors up to the square root of n because if n is divisible by any number greater than its square
11    for i in range(2, int(n**0.5) + 1):
12        # If n is divisible by any number in this range, it means n has a divisor other than 1 and itself, so we return False.
13        if n % i == 0:
14            return False
15    # If we find no divisors in the loop, then n is prime, and we return True.
16    return True
17
18 number = int(input("Enter a number: "))
19 if is_prime(number):
20     print(f"{number} is a prime number.")
21 else:
22     print(f"{number} is not a prime number.")

```

Output:



```

PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\OneDrive\Desktop\AI assisted coding\9.2.py\t2.py
Enter a number: 4
4 is not a prime number.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>

```

Observation:

AI-generated explanations for conditions and loops clarify the decision-making flow and iteration behavior within the program. These logical comments help users quickly understand how the program controls execution paths. As a result, debugging and code comprehension become more efficient.

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of an entire Python file.

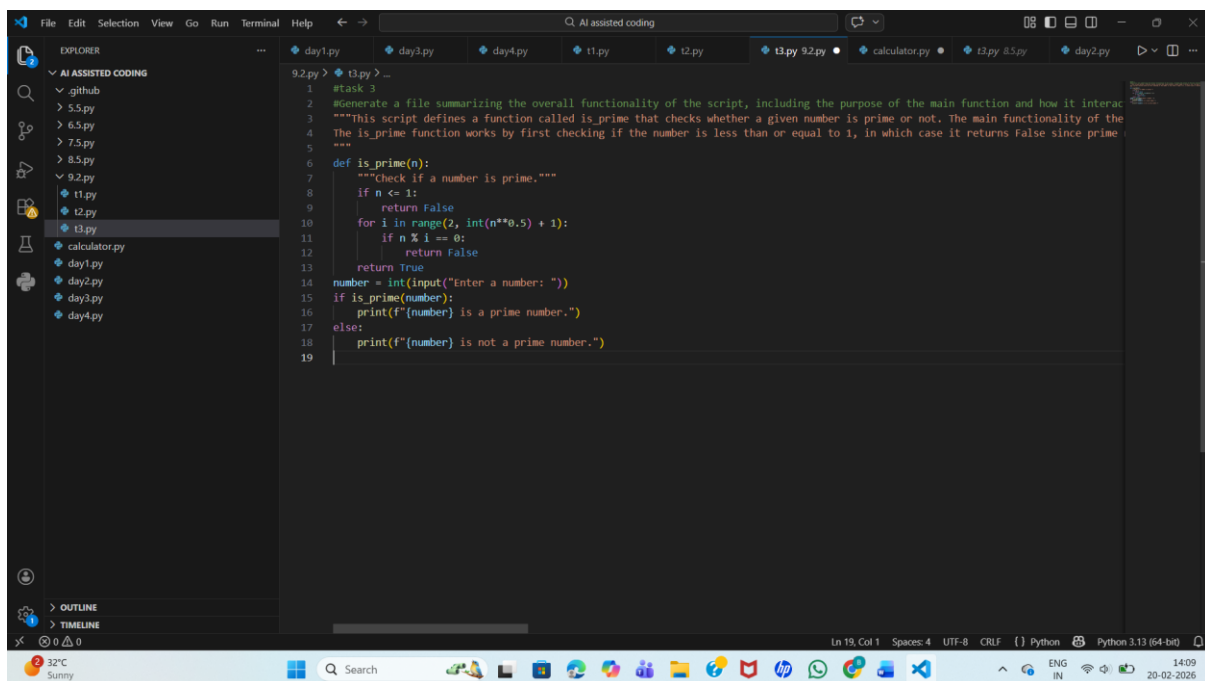
Instructions:

- Provide the complete Python file to AI.
- Ask AI to write a brief overview summarizing the file's purpose and functionality.
- Place the overview at the top of the file.

Expected Output -3:

A Python file with a clear and concise file-level overview at the beginning.

Code:



The screenshot shows a VS Code editor window with a Python file open. The file contains AI-generated documentation at the top, followed by a function definition and a main execution block. The documentation includes a task description and a brief overview of the file's purpose and functionality. The code is as follows:

```
1 #task 3
2 #Generate a file summarizing the overall functionality of the script, including the purpose of the main function and how it interacts with the user.
3 """This script defines a function called is_prime that checks whether a given number is prime or not. The main functionality of the script is to take a user input, check if it is a prime number, and print the result. The is_prime function works by first checking if the number is less than or equal to 1, in which case it returns False since prime numbers are greater than 1. If the number is greater than 1, it checks for factors by iterating from 2 to the square root of the number. If any factor is found, it returns False; otherwise, it returns True. The main function prompts the user to enter a number, checks if it is prime using the is_prime function, and prints the result accordingly. """
4
5
6 def is_prime(n):
7     """Check if a number is prime."""
8     if n <= 1:
9         return False
10    for i in range(2, int(n**0.5) + 1):
11        if n % i == 0:
12            return False
13    return True
14
15 number = int(input("Enter a number: "))
16 if is_prime(number):
17     print(f"{number} is a prime number.")
18 else:
19     print(f"{number} is not a prime number.")
```

Output:

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Us
ssisted coding/9.2.py/t3.py"
Enter a number: 6
6 is not a prime number.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> |
```

Observation:The AI successfully generated a concise file-level overview summarizing the overall purpose and functionality of the Python program. The overview was placed at the beginning of the file and described the program at a conceptual level without including implementation details. This improved the readability of the code and helped users quickly understand the program's objective.

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

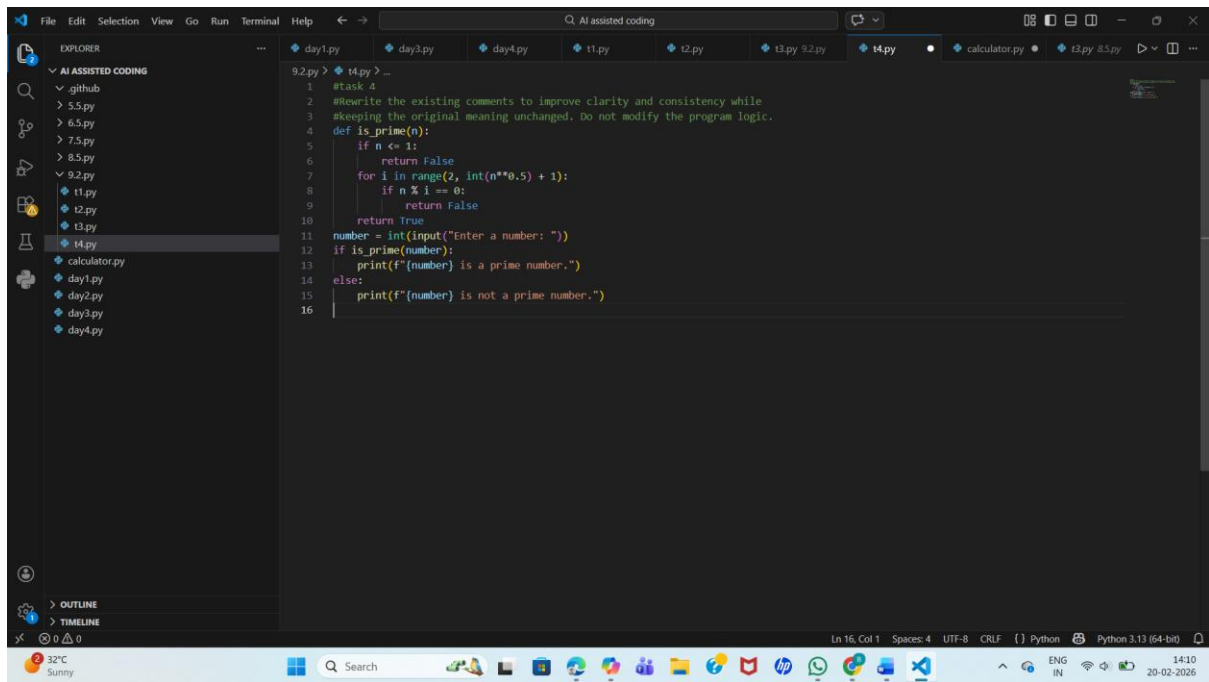
Instructions:

- Provide Python code containing basic or unclear comments.
- Ask AI to rewrite the documentation to improve clarity and consistency.
- Ensure technical meaning remains unchanged.

Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

Code:



Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> & C:\Users\abhin\OneDrive\Desktop\AI assisted coding\9.2.py\t4.py"
Enter a number: 9
9 is not a prime number.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> |
```

Observation:The AI improved the clarity and consistency of the existing comments without altering the program’s functionality. The refined documentation provided clearer explanations while preserving the original technical meaning. This enhancement made the code easier to read, understand, and maintain.

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

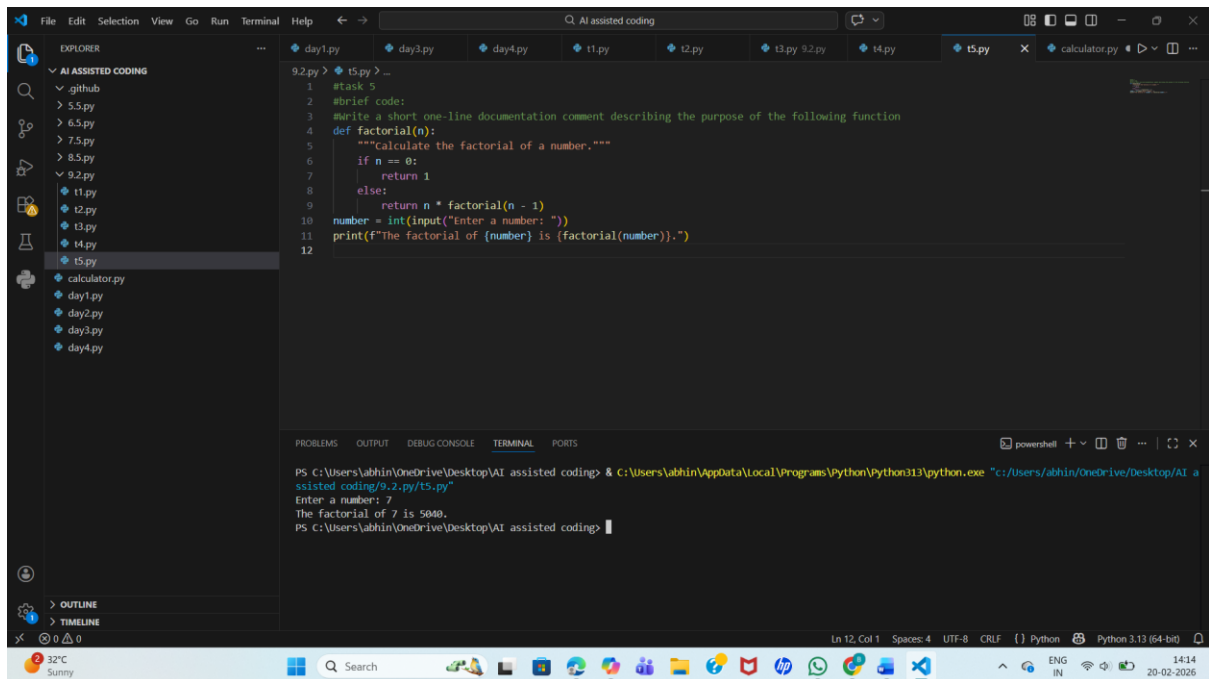
Create two prompts: one brief and one detailed.

- Use both prompts to document the same Python function.
- Compare the generated outputs.

Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

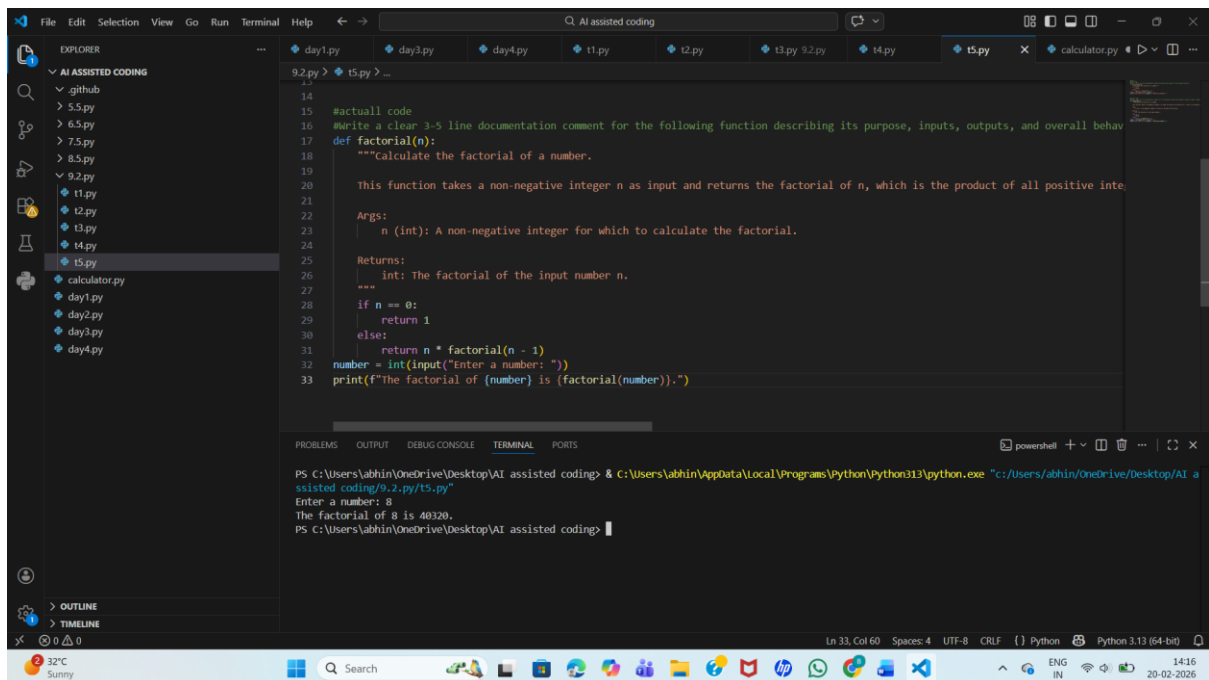
Code:



```
1 #task 5
2 #brief code:
3 #write a short one-line documentation comment describing the purpose of the following function
4 def factorial(n):
5     """calculate the factorial of a number."""
6     if n == 0:
7         return 1
8     else:
9         return n * factorial(n - 1)
10 number = int(input("Enter a number: "))
11 print(f"The factorial of {number} is {factorial(number)}.")
12
```

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> C:\Users\abhin\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/abhin/OneDrive/Desktop/AI assisted coding/9.2.py/t5.py"
Enter a number: 7
The factorial of 7 is 5040.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Actual code:



```
14
15 #actual code
16 #write a clear 3-5 line documentation comment for the following function describing its purpose, inputs, outputs, and overall behavior
17 def factorial(n):
18     """calculate the factorial of a number.
19
20     This function takes a non-negative integer n as input and returns the factorial of n, which is the product of all positive integers less than or equal to n.
21
22     Args:
23         n (int): A non-negative integer for which to calculate the factorial.
24
25     Returns:
26         int: The factorial of the input number n.
27     """
28     if n == 0:
29         return 1
30     else:
31         return n * factorial(n - 1)
32 number = int(input("Enter a number: "))
33 print(f"The factorial of {number} is {factorial(number)}.")
```

```
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding> C:\Users\abhin\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/abhin/OneDrive/Desktop/AI assisted coding/9.2.py/t5.py"
Enter a number: 8
The factorial of 8 is 40320.
PS C:\Users\abhin\OneDrive\Desktop\AI assisted coding>
```

Observation:

Criteria	Brief Prompt Output (One-line Comment)	Detailed Prompt Output (3–5 line Docstring)
Completeness	Provides only the basic purpose of the function.	Includes purpose, input description, output, and behavior details.
Clarity	Clear but very minimal explanation.	More descriptive, structured, and easier to understand.
Accuracy	Correct but limited technical information.	Highly accurate with full technical context and parameter details.